

API Reference

Complete guide to the portal REST API, with request and response examples

Generated on 2026-07-02

Internal document — valid for the current version of the portal

Table of Contents

1. API Introduction	4
1.1 Base URL	4
1.2 Response format	4
1.3 Authentication	4
1.4 Permissions	4
1.5 HTTP status codes used	5
2. Authentication	5
2.1 Login	5
2.2 Verify MFA code	6
2.3 Resend MFA code	6
2.4 Enable/disable two-step verification	7
2.5 E-mail service status (to allow MFA)	7
2.6 Own account data	7
2.7 Update own account	8
2.8 Change own password	8
2.9 Logout	8
3. Users (portal accounts)	9
3.1 List users	9
3.2 Create user	9
3.3 Change a user's role and modules	10
3.4 Delete user	10
4. Register (lookup lists)	10
4.1 List all register items	11
4.2 Create register item	11
4.3 Edit register item	12
4.4 Delete register item	12
5. Audit	12
5.1 Query audit trail	12
6. Settings	13
6.1 View e-mail (SMTP) configuration	13
6.2 Save e-mail configuration	13

6.3 Send test e-mail	14
6.4 View project identity and settings	14
6.5 Save project identity and settings	14
7. Dashboard	14
7.1 Public environment information	14
7.2 Aggregated KPIs	15
7.3 Export everything (JSON)	15
8. Reports	16
8.1 Send a module report by e-mail	16
9. Documentation (PDF downloads)	16
9.1 Portal user manual (Portuguese)	16
9.2 Portal user manual (English)	16
9.3 Database diagram and data dictionary (Portuguese)	16
9.4 Database diagram and data dictionary (English)	17
9.5 API Reference — this document (Portuguese)	17
9.6 API Reference — this document (English)	17
10. Data modules (generic CRUD)	17
10.1 Routes common to all modules	17
10.2 Access (acessos)	19
10.3 Changes (mudancas)	20
10.4 Backup (backup policies)	20
10.5 Restore (restore tests)	21
10.6 Data Dictionary (dicionario)	21
10.7 Integrations (integracoes)	22
11. Complete flow example (curl)	23

1. API Introduction

The portal exposes its own **HTTP/JSON API** (manual routing in PHP, no framework) that serves both the portal's own front-end and can be used by external scripts (integration routines, automations, data extraction for other tools). This reference documents all available routes, with request and response examples for each one.

1.1 Base URL

All routes are under the **/api** path, starting from the address where the portal is installed. If the portal is at the root of the domain, the base URL is:

```
https://YOUR-DOMAIN/api
```

If the portal was installed in a subfolder (e.g. *https://server/data-management*), the base URL includes that subfolder before **/api**:

```
https://YOUR-DOMAIN/data-management/api
```

Examples in this document use only "/api/..." — replace with your environment's domain (and subfolder, if any).

1.2 Response format

Every response is **JSON** (*Content-Type: application/json*), with Unicode characters preserved (not escaped) and without escaped slashes. Creation responses (POST) use status **201** and return the created record; deletions (DELETE of a single item) use **204**, with no body.

Errors always follow the same format across all routes:

Error format

```
{
  "detail": "Message explaining what went wrong."
}
```

1.3 Authentication

Most routes require an access token, sent in the **Authorization** header on every call:

```
Authorization: Bearer YOUR_TOKEN_HERE
```

The token is obtained via *POST /auth/login* (section 2.1) and expires after **480 minutes (8 hours)** by default — this duration is configurable by the administrator (*token_expire_minutes* in the portal configuration). A token is also immediately invalidated on logout or password change, even before it expires.

Only four routes work **without** a token: *GET /*, *POST /auth/login*, *POST /auth/mfa/verificar*, *POST /auth/mfa/reenviar*, and *GET /dashboard/info* — the last one intentionally, as the login screen shows the configured database engine before the user signs in.

1.4 Permissions

Beyond the token, each route may require a minimum **role** (Read, Write, Administrator, or Master) or access granted to a specific module. This is described individually for each route in this reference. In general:

- **Read:** may only perform GET on modules allowed for the user.
- **Write:** may perform GET/POST/PUT/DELETE on modules allowed for the user.
- **Administrator:** full access to all modules, plus Users, Register (Types), and Settings routes.
- **Master:** everything the Administrator can do, plus exclusive access to *GET /auditoria*.

1.5 HTTP status codes used

Code	Meaning
200	Success (query or update).
201	Record created successfully (response body contains the full record).
204	Success with no response body (e.g. single-item deletion).
400	Request rejected by a business rule (e.g. duplicate login).
401	Not authenticated — token missing, invalid, expired, or revoked.
403	Authenticated but not authorised for this action.
404	Record or route not found.
422	Invalid data in the request body (required field missing, invalid e-mail, etc.).
429	Too many consecutive login attempts — temporary anti-brute-force lockout.
500	Internal server error.

2. Authentication

Standard flow: *POST /auth/login* returns a token (or requests a second factor if MFA is active); that token goes in the *Authorization* header on all subsequent calls, until it expires or the user logs out.

2.1 Login

POST */auth/login*

No authentication required.

Body as *application/x-www-form-urlencoded* (not JSON) — same format as an HTML form:

Body (form-urlencoded)

```
username=ana.dba&password=StrongPassword123!
```

curl

```
curl -X POST https://YOUR-DOMAIN/api/auth/login \
  --data-urlencode 'username=ana.dba' \
  --data-urlencode 'password=StrongPassword123!'
```

Response 200 (no MFA)

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGc...",
  "token_type": "bearer"
}
```

If the user has two-step verification enabled, the response does not carry the access token yet — it carries a temporary *mfa_token*, and a 6-digit code is sent by e-mail:

Response 200 (MFA active)

```
{
  "mfa_required": true,
  "mfa_token": "8f3a1c9e..."
}
```

Invalid username/password → 401. Too many consecutive failed attempts (same user + same IP) → 429 with the remaining lockout time in the message.

2.2 Verify MFA code

POST /auth/mfa/verificar

No authentication required (uses the mfa_token).

Second step of login: checks the 6-digit code received by e-mail against the *mfa_token* returned in the previous step. Only here is the real access token issued.

Body (JSON)

```
{
  "mfa_token": "8f3a1c9e...",
  "codigo": "482915"
}
```

Response 200

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGc...",
  "token_type": "bearer"
}
```

Wrong code, expired code (10 minutes), or invalid mfa_token → 401.

2.3 Resend MFA code

POST /auth/mfa/reenviar

No authentication required (uses the previous mfa_token).

Generates and sends a new code by e-mail, invalidating the previous one. The *mfa_token* also changes — the front-end must update its stored value with the one returned here.

Body (JSON)

```
{
  "mfa_token": "8f3a1c9e..."
}
```

Response 200

```
{
  "mfa_token": "a17bd02f..."
}
```

2.4 Enable/disable two-step verification

PUT /auth/mfa

Requires token. Any user, on their own account.

Requires confirming the current password. Enabling (*ativo: true*) is only allowed if the user has an e-mail address on their profile and the portal's mail server has already been configured and tested (see section 6).

Body (JSON)

```
{
  "ativo": true,
  "senha_atual": "StrongPassword123!"
}
```

Response 200 (updated user, without password_hash)

```
{
  "id": 7,
  "username": "ana.dba",
  "nome_completo": "Ana Silva",
  "email": "ana@company.com",
  "role": "escrita",
  "mfa_ativo": 1,
  "...": "..."
}
```

Wrong current password → 400. Missing e-mail or untested mail service → 422.

2.5 E-mail service status (to allow MFA)

GET /auth/mfa/email-status

Requires token. Any logged-in user.

Response 200

```
{
  "pronto": true
}
```

2.6 Own account data

GET /auth/me

Requires token. Any logged-in user.

Response 200

```
{
  "id": 7,
  "username": "ana.dba",
  "nome_completo": "Ana Silva",
  "email": "ana@company.com",
  "role": "escrita",
  "modulos_permitidos": "acessos,mudancas",
  "cor_tema": "padrao",
  "estilo_side": "claro",
  "mfa_ativo": 0,
  "...": "..."
}
```

2.7 Update own account

PUT /auth/me

Requires token. Any user, on their own account.

Updates display name, e-mail, colour theme, and sidebar style. Does not change password, role, or allowed modules (those are managed by an administrator, sections 3 and below).

Body (JSON)

```
{
  "nome_completo": "Ana Silva",
  "email": "ana@company.com",
  "cor_tema": "azul",
  "estilo_side": "slate"
}
```

cor_tema accepts: padrao, azul, verde, violeta. estilo_side accepts: claro, slate. An invalid value silently falls back to the default.

2.8 Change own password

POST /auth/change-password

Requires token. Any user, on their own account.

Body (JSON)

```
{
  "senha_atual": "StrongPassword123!",
  "nova_senha": "AnotherStrongPwd456!"
}
```

Response 200

```
{
  "ok": true
}
```

On password change the current token is immediately revoked — the user must log in again. Password too weak → 400.

2.9 Logout

POST /auth/logout

Requires token.

Revokes the current token on the server (it becomes invalid even before it expires).

Response 200

```
{
  "ok": true
}
```

3. Users (portal accounts)

Management of accounts that access the portal. All routes in this section require the **Administrator** or **Master** role.

3.1 List users

GET /usuarios

Requires token + Administrator/Master role.

Response 200 (list, without password_hash)

```
[
  {
    "id": 1, "username": "master", "role": "master", "modulos_permitidos": "", "...": "..." },
  {
    "id": 7, "username": "ana.dba", "role": "escrita", "modulos_permitidos": "acessos,mudancas" }
]
```

3.2 Create user

POST /usuarios

Requires token + Administrator/Master role.

Body (JSON)

```
{
  "username": "joao.dba",
  "password": "StrongPassword789!",
  "nome_completo": "John Smith",
  "email": "joao@company.com",
  "role": "escrita",
  "modulos": ["acessos", "mudancas", "backup"]
}
```

Response 201 (without password_hash)

```
{
  "id": 12,
  "username": "joao.dba",
  "nome_completo": "John Smith",
  "email": "joao@company.com",
  "role": "escrita",
  "modulos_permitidos": "acessos,mudancas,backup",
  "...": "..."
}
```

"role" accepts leitura, escrita, or admin (invalid value falls back to leitura). "modulos" only applies to leitura/escrita — admin already has full access. Duplicate login → 400; weak password → 422.

3.3 Change a user's role and modules

PUT /usuarios/{id}/permissoes

Requires token + Administrator/Master role.

Body (JSON)

```
{
  "role": "escrita",
  "modulos": ["acessos", "mudancas", "integracoes"]
}
```

Response 200

```
{
  "id": 12,
  "username": "joao.dba",
  "role": "escrita",
  "modulos_permitidos": "acessos,mudancas,integracoes",
  "...": "..."
}
```

Only a Master user can change another Master's permissions → 403 otherwise. User not found → 404.

3.4 Delete user

DELETE /usuarios/{id}

Requires token + Administrator/Master role.

Response **204** (no body) on success.

Cannot delete the currently logged-in user → 400. Deleting another Master without being Master → 403. User not found → 404.

4. Register (lookup lists)

The lists used in the forms for Changes, Backup, Restore, Access, and Integrations (dropdowns such as Status, Criticality, Environment, etc.) are managed through a single generic route based on a *category*.

Category	Used in
mudanca_ambiente	Environment field, in Changes.

Category	Used in
mudanca_tipo	Type field, in Changes.
mudanca_status	Status field, in Changes.
backup_criticidade	Criticality field, in Backup.
backup_tipo	Backup type field, in Backup.
backup_resultado	Result field, in Restore.
acesso_nivel	Access level field, in Access.
integracao_tipo	Type field, in Integrations.
integracao_ambiente	Environment field, in Integrations.
integracao_criticidade	Criticality field, in Integrations.
integracao_status	Status field, in Integrations.

4.1 List all register items

GET /tipos

Requires token (any logged-in role).

Returns **all** items from **all** categories at once (the front-end filters by category locally); ordered by category and then by id.

Response 200

```
[
  { "id": 1, "categoria": "mudanca_status", "nome": "Planejada" },
  { "id": 2, "categoria": "mudanca_status", "nome": "Aprovada" }
]
```

4.2 Create register item

POST /tipos

Requires token + Administrator/Master role.

Body (JSON)

```
{
  "categoria": "integracao_status",
  "nome": "In staging"
}
```

Response 201

```
{
  "id": 34,
  "categoria": "integracao_status",
  "nome": "In staging"
}
```

Category outside the list above → 422. Empty name → 422. Duplicate name in the same category (case-insensitive comparison) → 400.

4.3 Edit register item

PUT /tipos/{id}

Requires token + Administrator/Master role.

Body (JSON)

```
{
  "nome": "In staging (QA)"
}
```

Response 200

```
{
  "id": 34,
  "categoria": "integracao_status",
  "nome": "In staging (QA)"
}
```

4.4 Delete register item

DELETE /tipos/{id}

Requires token + Administrator/Master role.

Response **204** (no body). Item not found → 404.

5. Audit

5.1 Query audit trail

GET /auditoria

Requires token + Master role (exclusive).

Lists creation/update/deletion events across all modules, as well as login, logout, password changes, permission changes, and MFA events. Accepts optional query-string filters:

Parameter	Description
usuario	Filter by the login of who performed the action.
tabela	Filter by the affected module/table (e.g. acessos, usuarios, auth).
acao	Filter by action type (e.g. criar, atualizar, excluir, login_sucesso).
inicio / fim	Date range (YYYY-MM-DD format).
pagina	Page number, starting at 1 (default: 1).
por_pagina	Items per page (default: 50).

Example URL

```
GET /api/auditoria?usuario=ana.dba&tabela=acessos&pagina=1&por_pagina=50
```

Response 200 (abbreviated format)

```
{
  "total": 132,
  "pagina": 1,
  "por_pagina": 50,
  "registros": [
    {
      "id": 980, "tabela": "acessos", "registro_id": 41, "acao": "atualizar",
      "usuario": "ana.dba", "ip": "10.0.0.14", "criado_em": "2026-06-29 14:02:11",
      "dados_antes": { "status": "Ativo" }, "dados_depois": { "status": "Revogado" }
    }
  ]
}
```

Access by a role other than Master → 403 ("Apenas o usuario master pode acessar a trilha de auditoria.").

6. Settings

6.1 View e-mail (SMTP) configuration

```
GET /config/email
```

Requires token + Administrator/Master role.

Response 200 (SMTP password not included)

```
{
  "host": "smtp.company.com",
  "porta": 587,
  "seguranca": "tls",
  "usuario": "no-reply@company.com",
  "remetente_nome": "Data Management Portal",
  "remetente_email": "no-reply@company.com",
  "testado_ok": true
}
```

6.2 Save e-mail configuration

```
PUT /config/email
```

Requires token + Administrator/Master role.

Body (JSON)

```
{
  "host": "smtp.company.com",
  "porta": 587,
  "seguranca": "tls",
  "usuario": "no-reply@company.com",
  "senha": "Smtppassword",
  "remetente_nome": "Data Management Portal",
  "remetente_email": "no-reply@company.com"
}
```

host and remetente_email are required → 422 if missing. remetente_email must be a valid e-mail → 422.

6.3 Send test e-mail

POST /config/email/testar

Requires token + Administrator/Master role.

Body (JSON)

```
{
  "para": "ana@company.com"
}
```

Response 200

```
{
  "ok": true
}
```

Send failure (wrong host, invalid credentials, etc.) → 400, with the SMTP error message. A successful test send is what marks the mail server as "testado_ok" and unlocks MFA activation for users (section 2.4).

6.4 View project identity and settings

GET /config/projeto

Requires token (any logged-in role).

Response 200

```
{
  "nome_projeto": "Data Management Portal",
  "logo_data": "data:image/png;base64,...",
  "titulo_efetivo": "Data Management Portal",
  "timeout_inatividade_min": 30
}
```

6.5 Save project identity and settings

PUT /config/projeto

Requires token + Administrator/Master role.

Body (JSON)

```
{
  "nome_projeto": "Data Management Portal",
  "timeout_inatividade_min": 60,
  "logo_data": "..."
}
```

timeout_inatividade_min must be between 5 and 480 → 422 outside that range. logo_data must be an image data URI (data:image/...) up to ~2 MB in base64 → 422 otherwise.

7. Dashboard

7.1 Public environment information

GET /dashboard/info

No authentication required.

The only data route open to the public (no token) — used by the login screen to display the configured database engine before the user signs in.

Response 200

```
{
  "motor_banco": "PostgreSQL",
  "versao": "1.0.0"
}
```

7.2 Aggregated KPIs

GET /dashboard/stats

Requires token. Numbers reflect only the modules allowed for the user.

Response 200

```
{
  "acessos_ativos": 58, "acessos_total": 64, "acessos_a_revisar": 5,
  "mudancas_30_dias": 12, "mudancas_pendentes": 3,
  "ultimo_restore": "2026-06-20", "dias_desde_ultimo_restore": 10,
  "bancos_sem_teste": ["ERP_PROD"], "bancos_com_politica": 6, "dados_sensiveis": 41,
  "totais": { "acessos": 64, "mudancas": 30, "backup": 6, "restore": 18,
    "dicionario": 410, "integracoes": 22 },
  "por_status_acessos": { "Ativo": 58, "Revogado": 6 },
  "por_status_mudancas": { "Executada": 22, "Planejada": 3 },
  "por_criticidade_backup": { "Alta": 4, "Media": 2 },
  "integracoes_total": 22, "integracoes_criticas": 3, "integracoes_sensiveis": 7,
  "integracoes_sem_responsavel": 1, "integracoes_revisao_vencida": 4
}
```

7.3 Export everything (JSON)

GET /dashboard/export

Requires token. Only returns modules allowed for the user.

Used by the *Export all (JSON)* button on the Reports screen — returns all records from all visible modules in a single file.

Response 200 (structure)

```
{
  "gerado_em": "2026-06-30",
  "motor_banco": "PostgreSQL",
  "acessos": [ { "id": 1, "usuario": "...", "...": "..." } ],
  "mudancas": [ ],
  "backup": [ ],
  "restore": [ ],
  "dicionario": [ ],
  "integracoes": [ ]
}
```

8. Reports

8.1 Send a module report by e-mail

POST /relatorios/{modulo}/email

Requires token + read access to the module.

{modulo} is one of the prefixes from section 10 (acessos, mudancas, backup, restore, dicionario, integracoes). Generates a CSV (semicolon separator, UTF-8 BOM) and sends it as an e-mail attachment.

Body (JSON) — inicio/fim are optional

```
{
  "para": "ana@company.com",
  "inicio": "2026-06-01",
  "fim": "2026-06-30"
}
```

Response 200

```
{
  "ok": true
}
```

inicio/fim filter by the internal criado_em column (date the record was added to the portal). No mail server configured → 400. Module not found → 404.

9. Documentation (PDF downloads)

These routes serve the portal manuals as PDFs, always through an authenticated API call (rather than a static file), so no web-server configuration for .pdf extension is needed.

9.1 Portal user manual (Portuguese)

GET /documentacao

Requires token.

Binary response (application/pdf), with Content-Disposition: attachment header — the browser downloads the file instead of displaying it.

9.2 Portal user manual (English)

GET /documentacao/manual-en

Requires token.

Same as above, English version.

9.3 Database diagram and data dictionary (Portuguese)

GET /documentacao/diagrama

Requires token.

PDF with the entity diagram and data dictionary of the portal's own database.

9.4 Database diagram and data dictionary (English)

```
GET /documentacao/diagrama-en
```

Requires token.

English version — table and column names remain in Portuguese (they are the actual DB names), but all descriptions are in English.

9.5 API Reference — this document (Portuguese)

```
GET /documentacao/api
```

Requires token.

Returns the Portuguese API manual as a PDF.

9.6 API Reference — this document (English)

```
GET /documentacao/api-en
```

Requires token.

Returns this document as a PDF.

On all routes above, if the file does not exist on the server the response is 404 with the corresponding message.

10. Data modules (generic CRUD)

The six data modules (Access, Changes, Backup, Restore, Dictionary, and Integrations) share exactly the same set of routes — only the *{modulo}* prefix and the accepted body fields change. The table below summarises each module's prefix, the column used to sort the listing (always descending), and the fields searched by the *q* parameter.

{modulo}	Sorted by	Search field (q)
acessos	data	usuario, servidor, objeto, nivel, justificativa, solicitante, aprovador, status
mudancas	data	codigo, descricao, solicitante, aprovador, status, ambiente
backup	id	banco, tipo, local, responsavel
restore	data	banco, backup, por, resultado
dicionario	id	servidor, banco, tabela, coluna, tipo_dado, descricao, classificacao
integracoes	nome	nome, origem, ip_origem, destino, ip_destino, tipo, mecanismo, resp_tecnico, resp_negocio, status

*Every record created by any of these modules automatically gets *id*, *criado_em*, and *atualizado_em* fields — there is no need (and no effect) to include them in the request body.*

10.1 Routes common to all modules

Examples below use the **acessos** module — the same pattern applies to the other five, changing only the prefix and body fields (see each module's field table in sections 10.2–10.7).

List (with search and date filter)

GET /acessos

Requires token + read access to the module.

Parameter (querystring)	Description
q	Search text (optional) — searches the fields in the "Search field" column of the table above.
inicio / fim	Optional date range (YYYY-MM-DD), filtering by the internal criado_em column.

Example URL

```
GET /api/acessos?q=db_owner&inicio=2026-06-01&fim=2026-06-30
```

Response 200

```
[
  {
    "id": 41, "data": "2026-06-15", "usuario": "joao.silva", "tipo": "Login SQL",
    "servidor": "SRV-SQL01", "objeto": "ERP_PROD", "nivel": "db_owner",
    "justificativa": "Access for financial module maintenance",
    "solicitante": "Finance Management", "aprovador": "ana.dba", "revisao": "2026-12-15",
    "status": "Ativo", "obs": null,
    "criado_em": "2026-06-15 09:30:00", "atualizado_em": "2026-06-15 09:30:00"
  }
]
```

Fetch a single record

GET /acessos/{id}

Requires token + read access to the module.

Response 200 matching one item from the list above; unknown id → 404.

Create record

POST /acessos

Requires token + write access to the module.

Body (JSON)

```
{
  "data": "2026-06-30", "usuario": "joao.silva", "tipo": "Login SQL",
  "servidor": "SRV-SQL01", "objeto": "ERP_PROD", "nivel": "db_owner",
  "justificativa": "Access for financial module maintenance",
  "solicitante": "Finance Management", "aprovador": "ana.dba",
  "revisao": "2026-12-30", "status": "Ativo"
}
```

Response **201** with the full record (same format as GET, already with id/criado_em/atualizado_em populated). No field is mandatory in the generic CRUD — missing fields simply store null — except in the Dictionary module (section 10.6), which has its own uniqueness constraint.

Edit record

PUT /acessos/{id}

Requires token + write access to the module.

Body same as POST, but only sent fields are updated (omitted fields keep their current value). Response 200 with the updated record. Unknown id → 404.

Body (JSON) — partial update example

```
{
  "status": "Revogado",
  "obs": "Access revoked after employee departure."
}
```

Delete a single record

DELETE /acessos/{id}

Requires token + write access to the module.

Response **204** (no body). Unknown id → 404.

Bulk delete

DELETE /acessos/lote

Requires token + Administrator/Master role.

Removes multiple records at once. Without a body, deletes up to 5,000 records (default batch); with *ids*, deletes exactly the specified ids.

Body (JSON) — delete specific ids

```
{
  "ids": [12, 13, 14]
}
```

Body (JSON) — delete up to 500 records (without choosing which)

```
{
  "tamanho": 500
}
```

Response 200

```
{
  "apagados": 3
}
```

In the Dictionary module, bulk deletion specifically requires the Master role (Administrator is not enough) — it is the riskiest operation in the portal, as it often involves thousands of rows imported from a CSV.

10.2 Access (acessos)

Prefix: **acessos**. Body fields (POST/PUT):

Field	Description
data	Grant date.

Field	Description
usuario	User/login that received the access.
tipo	Account type (Login SQL, Login Windows, AD Group).
servidor	Server where the access was granted.
objeto	Database/object that was granted.
nivel	Access level(s) (free text, e.g. "db_owner" or "db_owner, db_datareader").
justificativa	Reason for granting.
solicitante / aprovador	Who requested and who approved.
revisao	Scheduled review date.
status	Ativo (Active), Revogado (Revoked), or Suspenso (Suspended).
obs	Free-form notes.

10.3 Changes (mudancas)

Prefix: **/mudancas**. Body fields (POST/PUT):

Field	Description
codigo	Change ticket code (e.g. CHG-001).
data / ambiente	Date and environment (Produção, Homologação, Desenvolvimento).
tipo	Change category (list managed in Register).
descricao	What was changed.
script	Affected objects (free text, may have multiple lines).
rollback	Rollback plan.
solicitante / aprovador	Who requested and who approved.
status	Planejada, Aprovada, Executada, Revertida, or Cancelada.
resultado	Execution result/notes.

Example body (POST /mudancas)

```
{
  "codigo": "CHG-042", "data": "2026-07-02", "ambiente": "Producao", "tipo": "DDL",
  "descricao": "Add email_verified column to the users table",
  "script": "ALTER TABLE usuarios ADD email_verificado BIT DEFAULT 0",
  "rollback": "ALTER TABLE usuarios DROP COLUMN email_verificado",
  "solicitante": "Product Team", "aprovador": "ana.dba", "status": "Aprovada"
}
```

10.4 Backup (backup policies)

Prefix: **/backup** (physical table: *backup_politicas*). Body fields (POST/PUT):

Field	Description
banco / criticidade	Database covered and its criticality (Alta/High, Media/Medium, Baixa/Low).
tipo	Full, Full + Diff, Full + Log, or Full + Diff + Log.
frequencia / horario	Frequency and scheduled time.
retencao	Backup retention period.
local	Storage location.
rpo / rto	RPO and RTO targets.
responsavel	Person/team responsible.

Example body (POST /backup)

```
{
  "banco": "ERP_PROD", "criticidade": "Alta", "tipo": "Full + Log",
  "frequencia": "Daily", "horario": "02:00", "retencao": "30 days",
  "local": "Offsite storage + cloud", "rpo": "1 hour", "rto": "4 hours",
  "responsavel": "Infrastructure Team"
}
```

10.5 Restore (restore tests)

Prefix: **/restore** (physical table: *restore_testes*). Body fields (POST/PUT):

Field	Description
data / banco	Test date and database tested.
backup	Backup used in the test.
tempo	Time taken to restore.
resultado	OK, Falha (Failure), or OK com ressalvas (OK with caveats).
por	Who performed the test.
obs	Free-form notes.

Example body (POST /restore)

```
{
  "data": "2026-06-28", "banco": "ERP_PROD", "backup": "Full 2026-06-27 02:00",
  "tempo": "38 min", "resultado": "OK", "por": "ana.dba"
}
```

10.6 Data Dictionary (dicionario)

Prefix: **/dicionario** (physical table: *dicionario_dados*). Body fields (POST/PUT):

Field	Description
servidor / banco / schema_nome / tabela / coluna	Exact location of the column.

Field	Description
tipo_dado	Column type in the database (e.g. varchar(11)).
permite_nulo	Sim (Yes) or Nao (No).
descricao	What the column represents.
classificacao	Publica, Interna, Confidencial, Confidencial (PII), or Restrita.
origem	Data origin system.
obs	Free-form notes.

Example body (POST /dicionario)

```
{
  "servidor": "SRV-SQL01", "banco": "ERP_PROD", "schema_nome": "dbo",
  "tabela": "clientes", "coluna": "cpf", "tipo_dado": "varchar(11)",
  "permite_nulo": "Nao", "descricao": "Customer tax ID (digits only)",
  "classificacao": "Confidencial (PII)", "origem": "ERP system registration"
}
```

This is the only module with its own uniqueness validation: two rows cannot share the same combination of banco + schema_nome + tabela + coluna (case-insensitive) — violating this returns 400 on both POST and PUT.

10.7 Integrations (integracoes)

Prefix: **/integracoes**. Body fields (POST/PUT):

Field	Description
nome	Integration/connection name.
origem / ip_origem	Source system and source IP.
destino / ip_destino	Destination system and destination IP.
tipo	Integration category (list managed in Register).
direcao	Entrada (Inbound), Saida (Outbound), or Bidirecional (Bidirectional).
mecanismo	E.g. REST API, ETL, database link, replication.
frequencia	Real-time, Every minute, Hourly, Daily, Weekly, Monthly, On demand, or Occasional.
dados_trafegados	Description of what is transmitted.
classificacao	Publico, Interno, Confidencial, or Personal/Sensitive (LGPD).
criticidade	Risk level (list managed in Register).
resp_tecnico / resp_negocio	Technical and business owners.
ambiente	Production, Staging, Development, etc. (list managed in Register).
status	Current status (list managed in Register).
ultima_revisao	Date of last review.
obs	Free-form notes.

Example body (POST /integracoes)

```
{
  "nome": "ERP -> Data Warehouse", "origem": "ERP_PROD", "ip_origem": "10.0.1.10",
  "destino": "DW_CORP", "ip_destino": "10.0.2.20", "tipo": "ETL",
  "direcao": "Saida", "mecanismo": "Nightly ETL job", "frequencia": "Daily",
  "dados_trafegados": "Orders, customers, and sales line items",
  "classificacao": "Personal / Sensitive (LGPD)", "criticidade": "Critical",
  "resp_tecnico": "ana.dba", "resp_negocio": "BI Management", "ambiente": "Producao",
  "status": "Ativa", "ultima_revisao": "2026-05-10"
}
```

11. Complete flow example (curl)

End-to-end walkthrough: login, list, create, edit, delete, and logout — useful as a starting point for an integration script.

Shell (bash) -- requires curl + jq

```
# 1) Login -- store the token in a variable
TOKEN=$(curl -s -X POST https://YOUR-DOMAIN/api/auth/login \
  --data-urlencode 'username=ana.dba' \
  --data-urlencode 'password=StrongPassword123!' | jq -r '.access_token')

# 2) List active access records
curl -s "https://YOUR-DOMAIN/api/acessos?q=Ativo" \
  -H "Authorization: Bearer $TOKEN"

# 3) Create a new access record
NOVO_ID=$(curl -s -X POST https://YOUR-DOMAIN/api/acessos \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"data":"2026-06-30","usuario":"joao.silva","servidor":"SRV-SQL01",
    "objeto":"ERP_PROD","nivel":"db_owner","status":"Ativo"}' | jq -r '.id')

# 4) Edit the record just created
curl -s -X PUT "https://YOUR-DOMAIN/api/acessos/$NOVO_ID" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"status":"Revogado"}'

# 5) Delete the record
curl -s -X DELETE "https://YOUR-DOMAIN/api/acessos/$NOVO_ID" \
  -H "Authorization: Bearer $TOKEN"

# 6) Logout
curl -s -X POST https://YOUR-DOMAIN/api/auth/logout \
  -H "Authorization: Bearer $TOKEN"
```

In production, never put usernames/passwords as plain text in a versioned script — use environment variables or a secrets manager.